

1.3.2 Circuit Switching

There are two fundamental approaches to moving data through a network of links and switches: **circuit switching** and **packet switching**.

Having covered packet-switched networks in the previous subsection,

In circuit-switched networks, the resources needed along a path (buffers, link transmission rate) to provide for communication between the end systems are *reserved* for the duration of the communication session between the end systems.

In packet-switched networks, these resources are *not* reserved;

As a simple analogy, consider two restaurants, one that requires reservations and another that neither requires reservations nor accepts them.

For the restaurant that requires reservations, we have to go through the hassle of calling before we leave home.

But when we arrive at the restaurant we can, in principle, immediately be seated and order our meal. For the restaurant that does not require reservations, we don't need to bother to reserve a table.

But when we arrive at the restaurant, we may have to wait for a table before we can be seated.

Traditional telephone networks are examples of circuit-switched networks.

Multiplexing in Circuit-Switched Networks

A circuit in a link is implemented with either **frequency-division multiplexing (FDM)** or **time-division multiplexing (TDM)**.

Packet Switching Versus Circuit Switching

Critics of packet switching have often argued that packet switching is not suitable for real-time services (for example, telephone calls and video conference calls) because of its variable and unpredictable end-to-end delays (due primarily to variable and unpredictable queuing delays).

Proponents of packet switching argue that

- (1) it offers better sharing of transmission capacity than circuit switching and
- (2) it is simpler, more efficient, and less costly to implement than circuit switching.

Generally speaking, people who do not like to hassle with restaurant reservations prefer packet switching to circuit switching.

Although packet switching and circuit switching are both prevalent in today's telecommunication networks, the trend has certainly been in the direction of packet switching.

Even many of today's circuit-switched telephone networks are slowly migrating toward packet switching.

In particular, telephone networks often use packet switching for the expensive overseas portion of a telephone call.

1.3.3 A Network of Networks

We saw earlier that end systems (PCs, smartphones, Web servers, mail servers, and so on) connect into the Internet via an access ISP.

The access ISP can provide either wired or wireless connectivity, using an array of access technologies including DSL, cable, FTTH, Wi-Fi, and cellular.

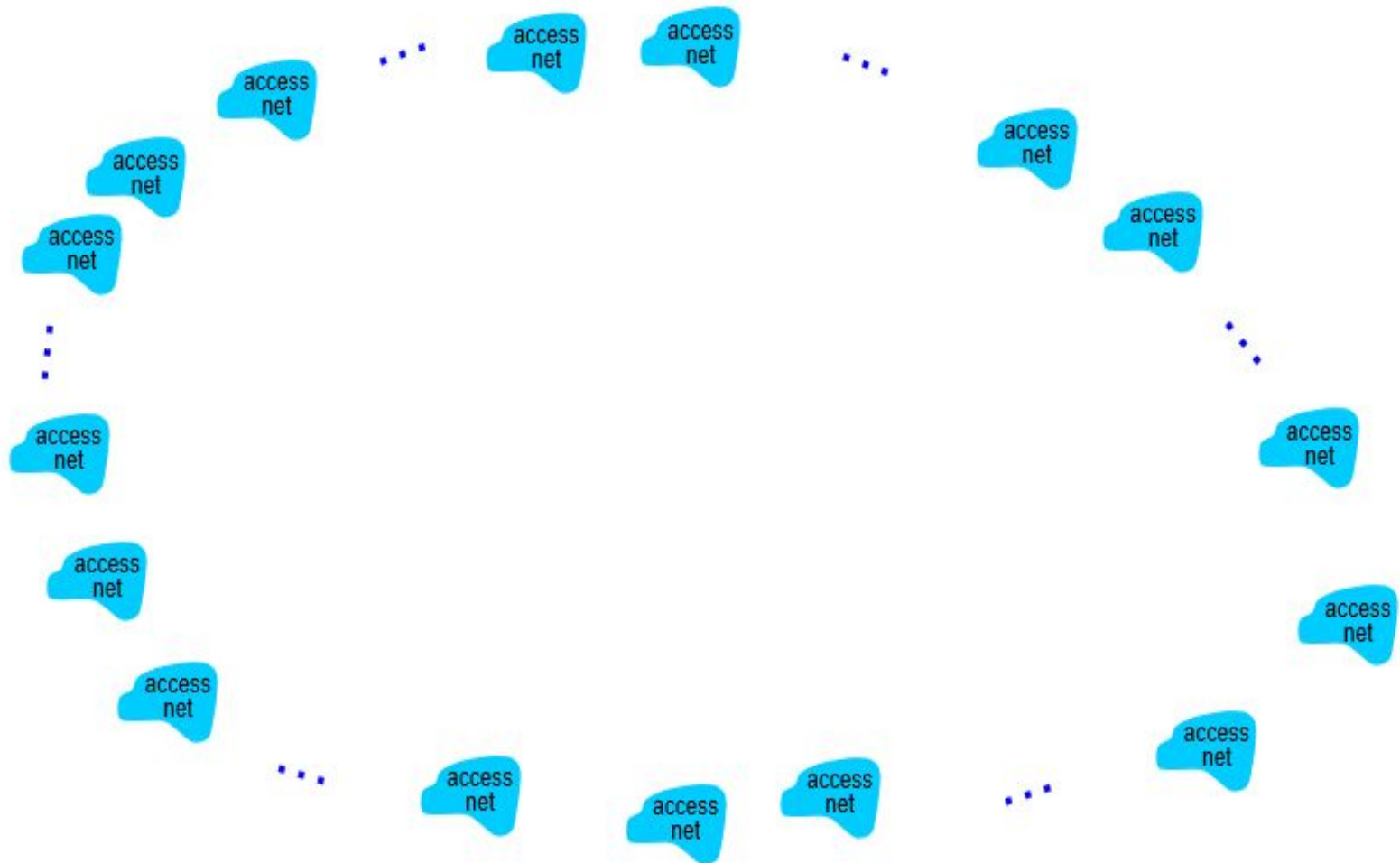
Note that the access ISP does not have to be a telco or a cable company; instead it can be, for example, a university (providing Internet access to students, staff, and faculty), or a company (providing access for its employees).

But connecting end users and content providers into an access ISP is only a small piece of solving the puzzle of connecting the billions of end systems that make up the Internet.

To complete this puzzle, the access ISPs themselves must be interconnected.

This is done by creating a *network of networks*—understanding this phrase is the key to understanding the Internet.

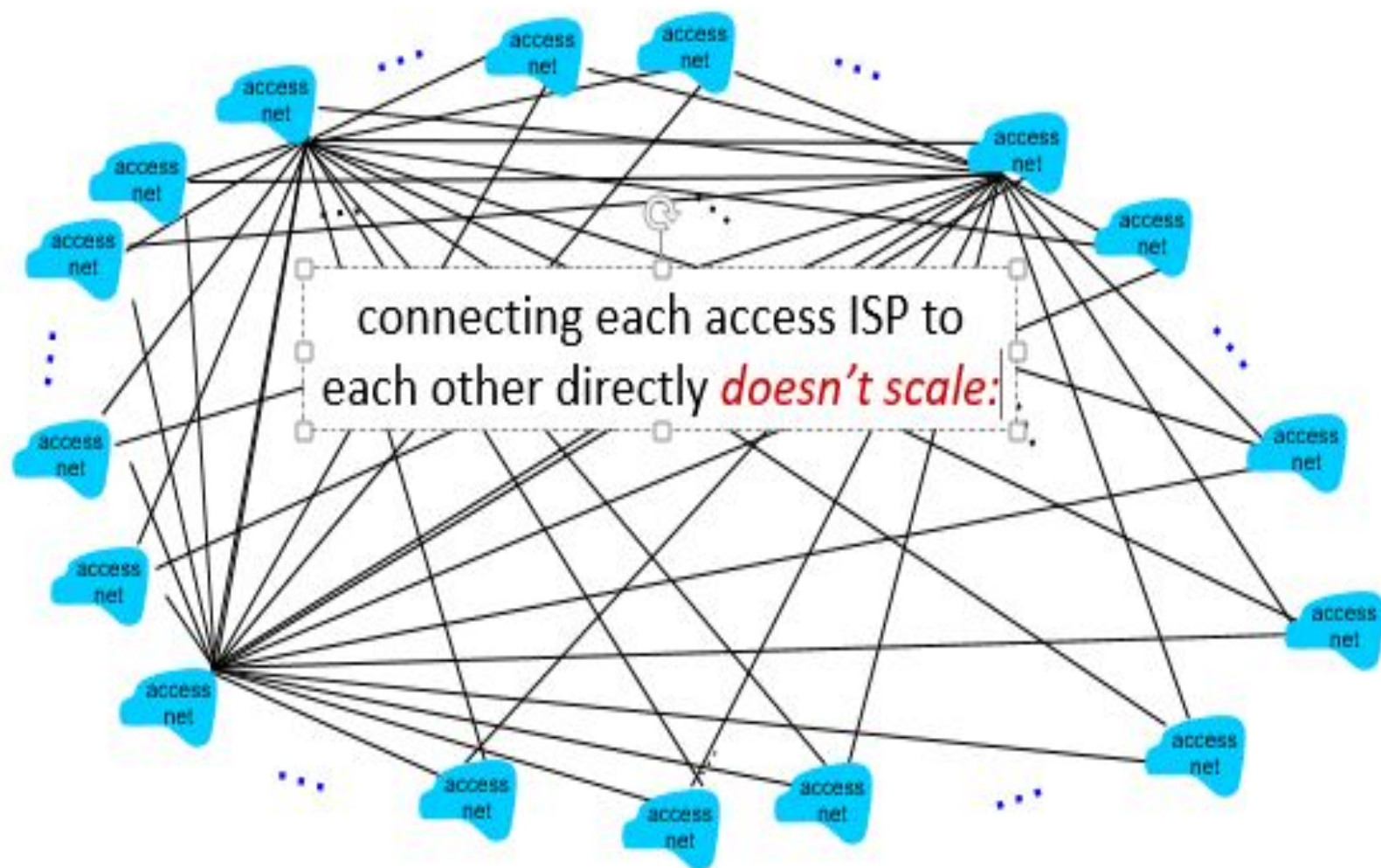
Question: given *millions* of access ISPs, how to connect them together?



In order to understand today's Internet network structure, let's incrementally build a series of network structures, with each new structure being a better approximation of the complex Internet that we have today.

Recall that the overarching goal is to interconnect the access ISPs so that all end systems can send packets to each other.

One naive approach would be to have each access ISP *directly* connect with every other access ISP. Such a mesh design is, of course, much too costly for the access ISPs, as it would require each access ISP to have a separate communication link to each of the hundreds of thousands of other access ISPs all over the world.

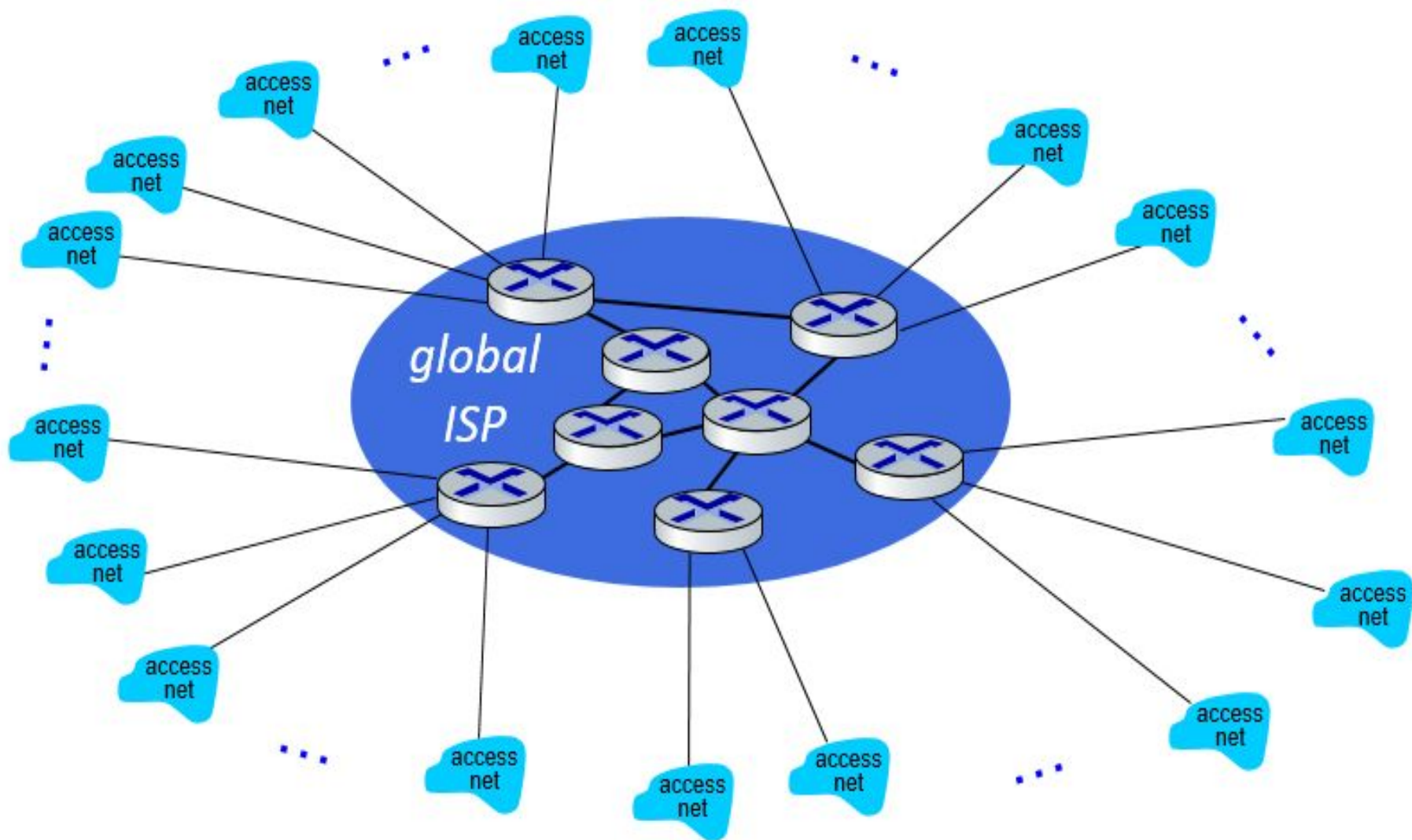


Our first network structure, *Network Structure 1*, interconnects all of the access ISPs with a *single global transit ISP*.

Our (imaginary) global transit ISP is a network of routers and communication links that not only spans the globe, but also has at least one router near each of the hundreds of thousands of access ISPs.

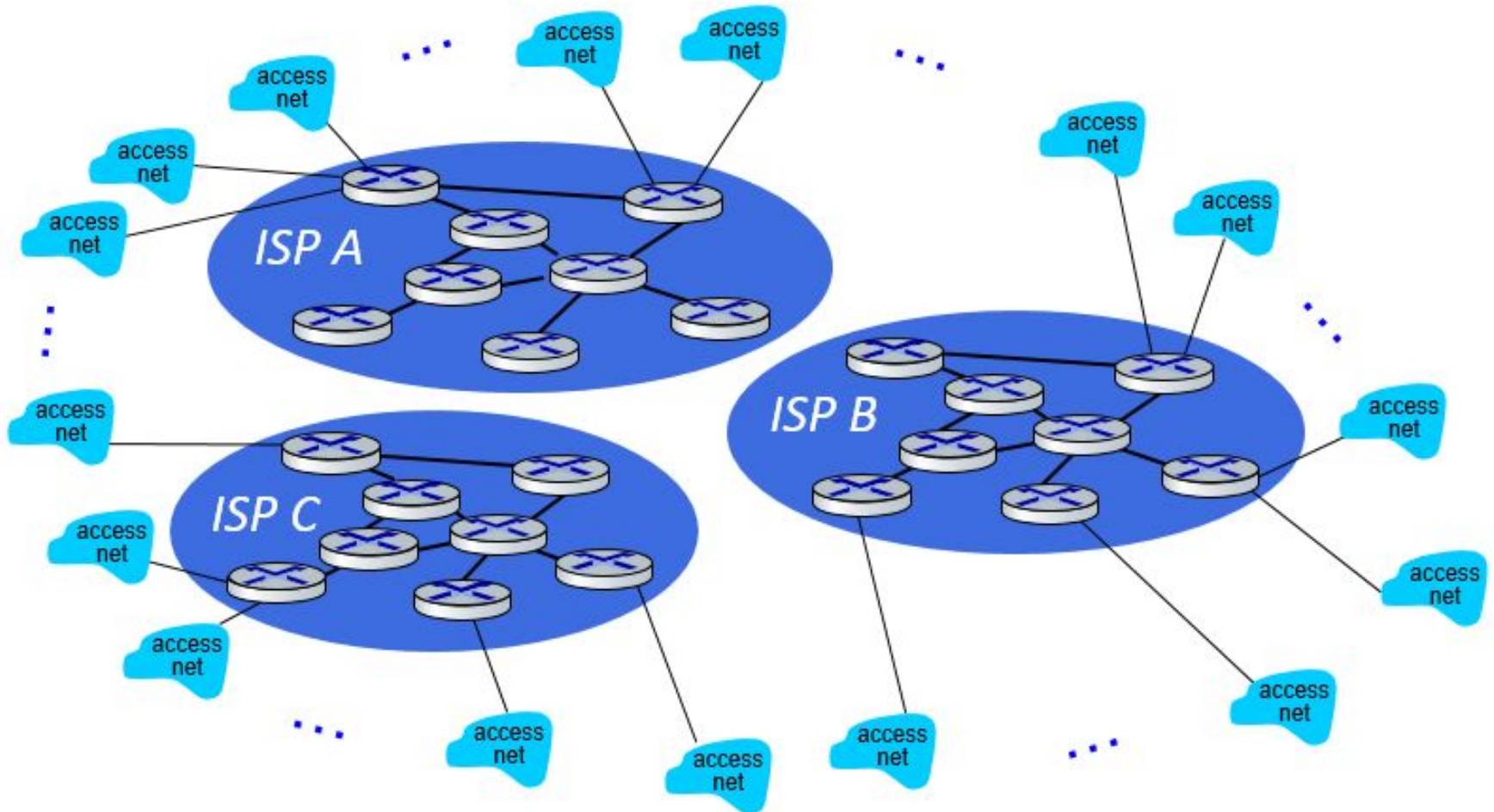
Of course, it would be very costly for the global ISP to build such an extensive network. To be profitable, it would naturally charge each of the access ISPs for connectivity, with the pricing reflecting (but not necessarily directly proportional to) the amount of traffic an access ISP exchanges with the global ISP.

Since the access ISP pays the global transit ISP, the access ISP is said to be a **customer** and the global transit ISP is said to be a **provider**.



Now if some company builds and operates a global transit ISP that is profitable, then it is natural for other companies to build their own global transit ISPs and compete with the original global transit ISP.

This leads to *Network Structure 2*, which consists of the hundreds of thousands of access ISPs and *multiple* global transit ISPs.



The access ISPs certainly prefer Network Structure 2 over Network Structure 1 since they can now choose among the competing global transit providers as a function of their pricing and services.

Note, however, that the global transit ISPs themselves must interconnect: Otherwise access ISPs connected to one of the global transit providers would not be able to communicate with access ISPs connected to the other global transit providers.

Network Structure 2, just described, is a **two-tier hierarchy** with global transit providers residing at the top tier and access ISPs at the bottom tier.

In reality, although some ISPs do have impressive global coverage and do directly connect with many access ISPs, no ISP has presence in each and every city in the world.

Instead, in any given region, there may be a **regional ISP** to which the access ISPs in the region connect.

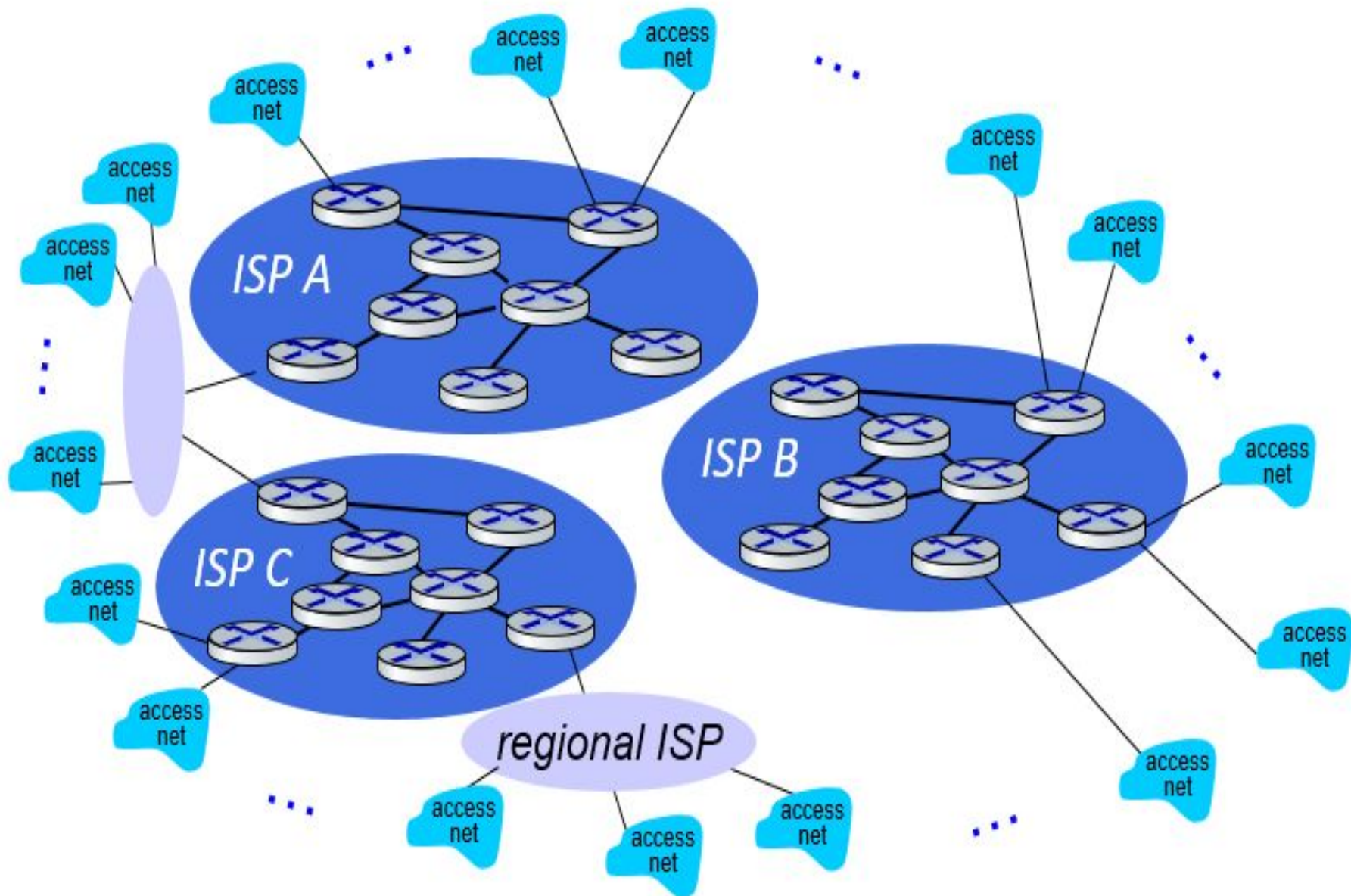
Each regional ISP then connects to **tier-1 ISPs**.

Tier-1 ISPs are similar to our (imaginary) global transit ISP; but tier-1 ISPs, which actually do exist, do not have a presence in every city in the world.

There are approximately a dozen tier-1 ISPs, including Level 3 Communications, AT&T, Sprint, and NTT.

Returning to this network of networks, not only are there multiple competing tier-1 ISPs, there may be multiple competing regional ISPs in a region.

In such a hierarchy, each access ISP pays the regional ISP to which it connects, and each regional ISP pays the tier-1 ISP to which it connects.



(An access ISP can also connect directly to a tier-1 ISP, in which case it pays the tier-1 ISP).

Thus, there is customer-provider relationship at each level of the hierarchy.

Note that the tier-1 ISPs do not pay anyone as they are at the top of the hierarchy.

To further complicate matters, in some regions, there may be a larger regional ISP (possibly spanning an entire country) to which the smaller regional ISPs in that region connect; the larger regional ISP then connects to a tier-1 ISP.

We refer to this **multi-tier hierarchy**, which is still only a crude approximation of today's Internet, as *Network Structure 3*.

To build a network that more closely resembles today's Internet, we must add points of presence (PoPs), multi-homing, peering, and Internet exchange points (IXPs) to the hierarchical Network Structure 3.

A **PoP** is simply a group of one or more routers in the provider's network where customer ISPs can connect into the provider ISP.

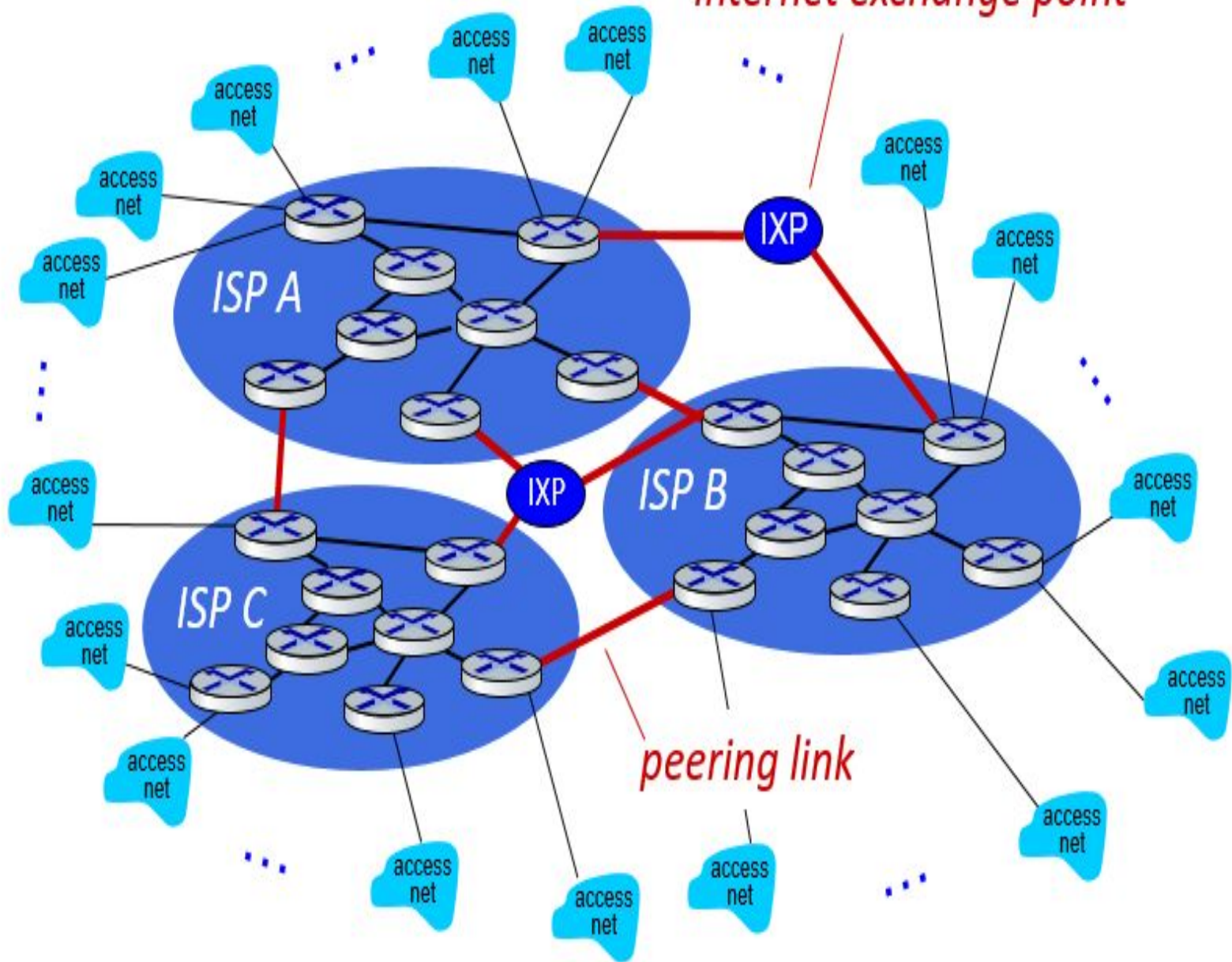
Any ISP (except for tier-1 ISPs) may choose to **multi-home**, that is, to connect to two or more provider ISPs.

So, for example, an access ISP may multi-home with two regional ISPs, or it may multi-home with two regional ISPs and also with a tier-1 ISP.

Similarly, a regional ISP may multi-home with multiple tier-1 ISPs.

When an ISP multi-homes, it can continue to send and receive packets into the Internet even if one of its providers has a failure.

Internet exchange point



As we just learned, customer ISPs pay their provider ISPs to obtain global Internet interconnectivity. The amount that a customer ISP pays a provider ISP reflects the amount of traffic it exchanges with the provider.

To reduce these costs, a pair of nearby ISPs at the same level of the hierarchy can **peer**, that is, they can directly connect their networks together so that all the traffic between them passes over the direct connection rather than through upstream intermediaries.

When two ISPs peer, it is typically settlement-free, that is, neither ISP pays the other.

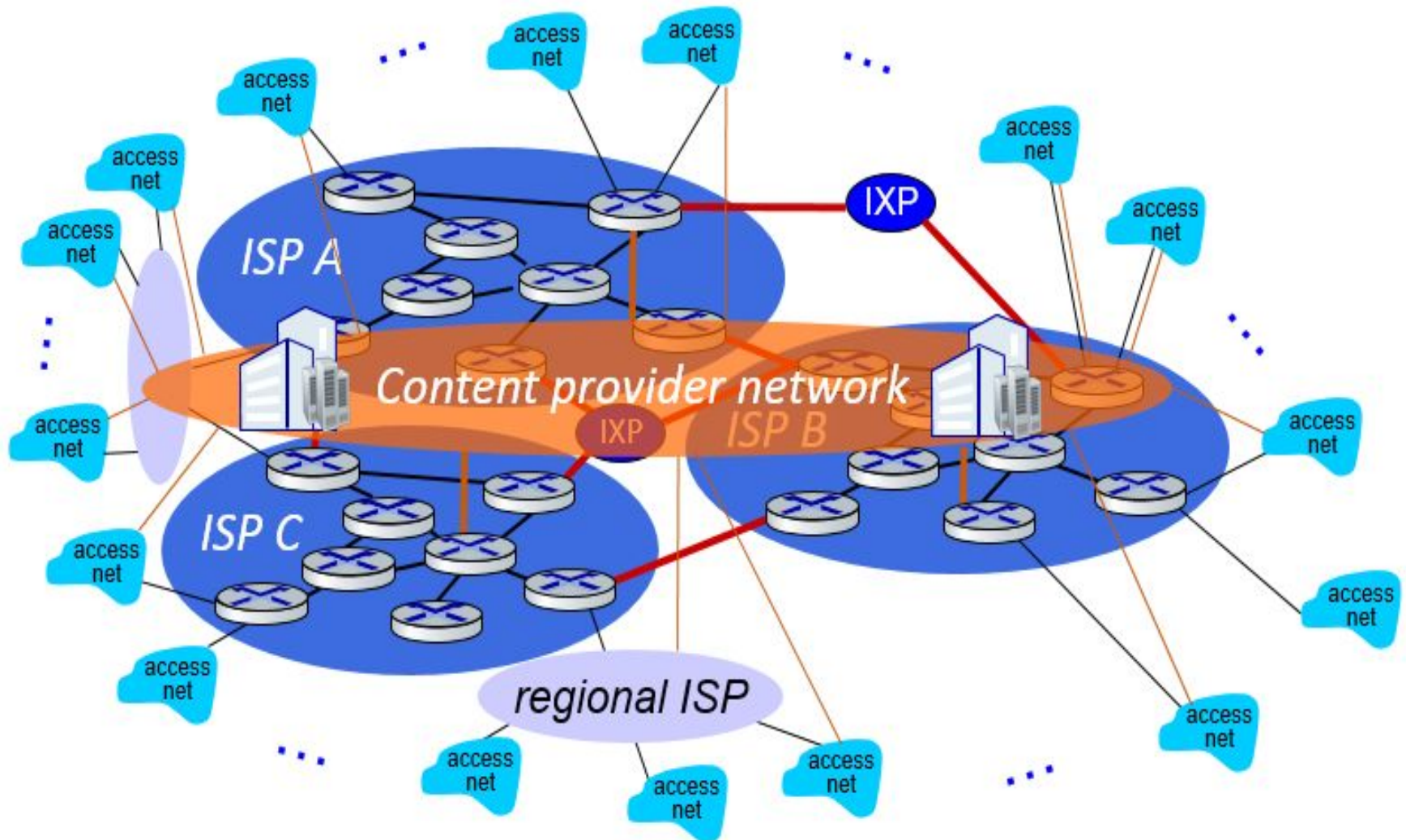
As noted earlier, tier-1 ISPs also peer with one another, settlement-free.

a third-party company can create an **Internet Exchange Point (IXP)**, which is a meeting point where multiple ISPs can peer together.

An IXP is typically in a stand-alone building with its own switches [Ager 2012].

We refer to this ecosystem—consisting of access ISPs, regional ISPs, tier-1 ISPs, PoPs, multi-homing, peering, and IXPs—as *Network Structure 4*.

We now finally arrive at *Network Structure 5*, which describes today's Internet. Network Structure 5, illustrated in Figure 1.15, builds on top of Network Structure 4 by adding **content-provider networks**.



Google is currently one of the leading examples of such a content-provider network.

As of this writing, it Google has 19 major data centers distributed across North America, Europe, Asia, South America, and Australia with each data center having tens or hundreds of thousands of servers.

The Google data centers are all interconnected via Google's private TCP/IP network, which spans the entire globe but is nevertheless separate from the public Internet.

Importantly, the Google private network only carries traffic to/from Google servers.

As shown in Figure 1.15, the Google private network attempts to “bypass” the upper tiers of the Internet by peering (settlement free) with lower-tier ISPs, either by directly connecting with them or by connecting with them at IXPs [Labovitz 2010].

However, because many access ISPs can still only be reached by transiting through tier-1 networks, the Google network also connects to tier-1 ISPs, and pays those ISPs for the traffic it exchanges with them.

By creating its own network, a content provider not only reduces its payments to upper-tier ISPs, but also has greater control of how its services are ultimately delivered to end users.

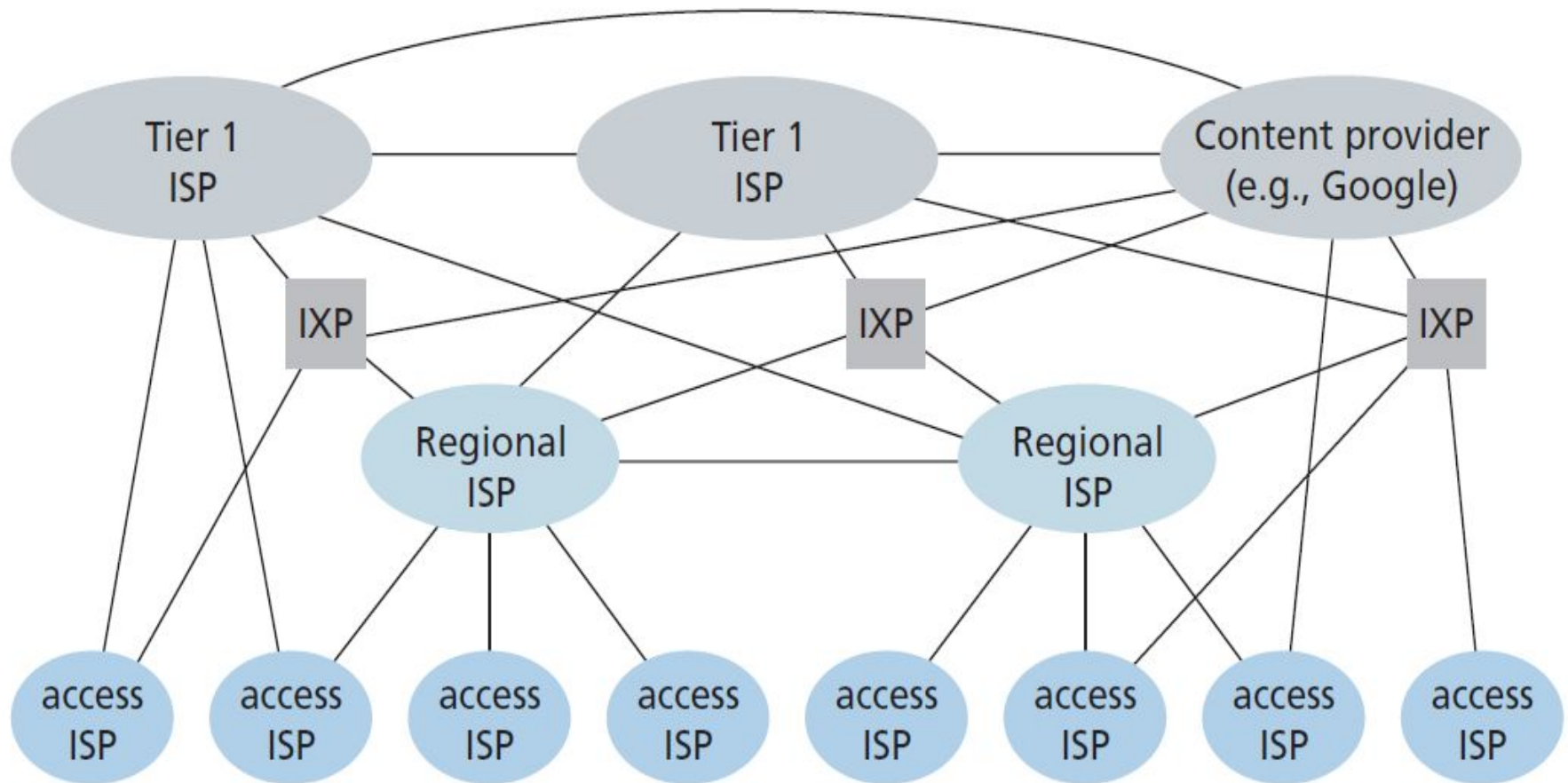


Figure 1.15 ♦ Interconnection of ISPs

1.4 Delay, Loss, and Throughput in Packet-Switched Networks

1.4.1 Overview of Delay in Packet-Switched Networks

Recall that a packet starts in a host (the source), passes through a series of routers, and ends its journey in another host (the destination).

As a packet travels from one node (host or router) to the subsequent node (host or router) along this path, the packet suffers from several types of delays at *each* node along the path. The most important of these delays are the **nodal processing delay**, **queuing delay**, **transmission delay**, and **propagation delay**; together, these delays accumulate to give a **total nodal delay**.

The performance of many Internet applications—such as search, Web browsing, e-mail, maps, instant messaging, and voice-over-IP—are greatly affected by network delays.

In order to acquire a deep understanding of packet switching and computer networks, we must understand the nature and importance of these delays.

Types of Delay

Let's explore these delays in the context of Figure 1.16.

As part of its end-to-end route between source and destination, a packet is sent from the upstream node through router A to router B.

Our goal is to characterize the nodal delay at router A.

Note that router A has an outbound link leading to router B.

This link is preceded by a queue (also known as a buffer).

When the packet arrives at router A from the upstream node, router A examines the packet's header to determine the appropriate outbound link for the packet and then directs the packet to this link.

In this example, the outbound link for the packet is the one that leads to router B.

A packet can be transmitted on a link only if there is no other packet currently being transmitted on the link and if there are no other packets preceding it in the queue; if the link is currently busy or if there are other packets already queued for the link, the newly arriving packet will then join the queue.

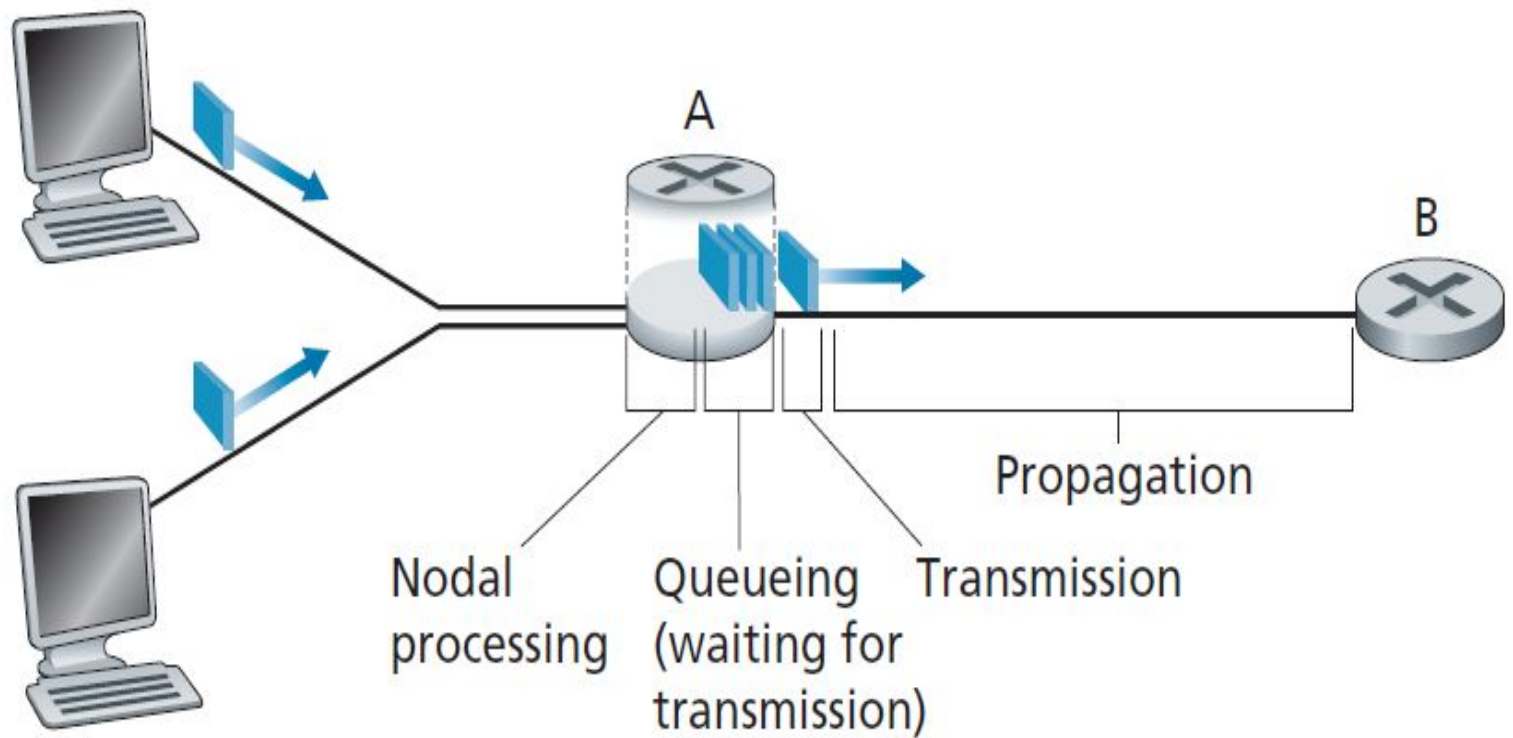


Figure 1.16 ♦ The nodal delay at router A

Processing Delay

The time required to examine the packet's header and determine where to direct the packet is part of the **processing delay**.

The processing delay can also include other factors, such as the time needed to check for bit-level errors in the packet that occurred in transmitting the packet's bits from the upstream node to router A.

Processing delays in high-speed routers are typically on the order of microseconds or less. After this nodal processing, the router directs the packet to the queue that precedes the link to router B.

Queuing Delay

At the queue, the packet experiences a **queuing delay** as it waits to be transmitted onto the link.

The length of the queuing delay of a specific packet will depend on the number of earlier-arriving packets that are queued and waiting for transmission onto the link.

If the queue is empty and no other packet is currently being transmitted, then our packet's queuing delay will be zero.

On the other hand, if the traffic is heavy and many other packets are also waiting to be transmitted, the queuing delay will be long.

Queuing delays can be on the order of microseconds to milliseconds in practice.

Transmission Delay

Assuming that packets are transmitted in a first-come-first-served manner, as is common in packet-switched networks, our packet can be transmitted only after all the packets that have arrived before it have been transmitted.

Denote the length of the packet by L bits, and denote the transmission rate of the link from router A to router B by R bits/sec.

For example, for a 10 Mbps Ethernet link, the rate is $R = 10$ Mbps; for a 100 Mbps Ethernet link, the rate is $R = 100$ Mbps.

The **transmission delay** is L/R .

This is the amount of time required to push (that is, transmit) all of the packet's bits into the link.

Transmission delays are typically on the order of microseconds to milliseconds in practice.

Propagation Delay

Once a bit is pushed into the link, it needs to propagate to router B.

The time required to propagate from the beginning of the link to router B is the **propagation delay**. The bit propagates at the propagation speed of the link.

The propagation speed depends on the physical medium of the link (that is, fiber optics, twisted-pair copper wire, and so on) and is in the range of

$$2 \cdot 10^8 \text{ meters/sec to } 3 \cdot 10^8 \text{ meters/sec}$$

which is equal to, or a little less than, the speed of light.

The propagation delay is the distance between two routers divided by the propagation speed.

That is, the propagation delay is d/s , where d is the distance between router A and router B and s is the propagation speed of the link.

Once the last bit of the packet propagates to node B, it and all the preceding bits of the packet are stored in router B.

The whole process then continues with router B now performing the forwarding.

In wide-area networks, propagation delays are on the order of milliseconds.

If we let d_{proc} , d_{queue} , d_{trans} , and d_{prop} denote the processing, queuing, transmission, and propagation delays, then the total nodal delay is given by

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

The contribution of these delay components can vary significantly.

For example, d_{prop} can be negligible (for example, a couple of microseconds) for a link connecting two routers on the same university campus; however, d_{prop} is hundreds of milliseconds for two routers interconnected by a geostationary satellite link, and can be the dominant term in d_{nodal} .

Similarly, d_{trans} can range from negligible to significant.

The processing delay, d_{proc} , is often negligible;

1.4.2 Queuing Delay and Packet Loss

The most complicated and interesting component of nodal delay is the queuing delay, d_{queue} .

Unlike the other three delays (namely, d_{proc} , d_{trans} , and d_{prop}), the queuing delay can vary from packet to packet.

For example, if 10 packets arrive at an empty queue at the same time, the first packet transmitted will suffer no queuing delay, while the last packet transmitted will suffer a relatively large queuing delay (while it waits for the other nine packets to be transmitted).

When is the queuing delay large and when is it insignificant? The answer to this question depends on the rate at which traffic arrives at the queue, the transmission rate of the link, and the nature of the arriving traffic, that is, whether the traffic arrives periodically or arrives in bursts.

To gain some insight here, let a denote the average rate at which packets arrive at the queue (a is in units of packets/sec).

Recall that R is the transmission rate; that is, it is the rate (in bits/sec) at which bits are pushed out of the queue.

The ratio La/R , called the **traffic intensity**, often plays an important role in estimating the extent of the queuing delay. estimating the extent of the queuing delay.

If $La/R > 1$, then the average rate at which bits arrive at the queue exceeds the rate at which the bits can be transmitted from the queue.

In this unfortunate situation, the queue will tend to increase without bound and the queuing delay will approach infinity! Therefore, one of the golden rules in traffic engineering is:

Design your system so that the traffic intensity is no greater than 1.

Now consider the case

$$La/R \leq 1$$

For example, if packets arrive periodically—that is, one packet arrives every L/R seconds—then every packet will arrive at an empty queue and there will be no queuing delay. On the other hand, if packets arrive in bursts but periodically, there can be a significant average queuing delay.

For example, suppose N packets arrive simultaneously every $(L/R)N$ seconds.

Then the first packet transmitted has no queuing delay; the second packet transmitted has a queuing delay of L/R seconds; and more generally, the n th packet transmitted has a queuing delay of $(n - 1)L/R$ seconds.

In particular, if the traffic intensity is close to zero, then packet arrivals are few and far between and it is unlikely that an arriving packet will find another packet in the queue. Hence, the average queuing delay will be close to zero.

One important aspect of Figure 1.18 is the fact that as the traffic intensity approaches 1, the average queuing delay increases rapidly.

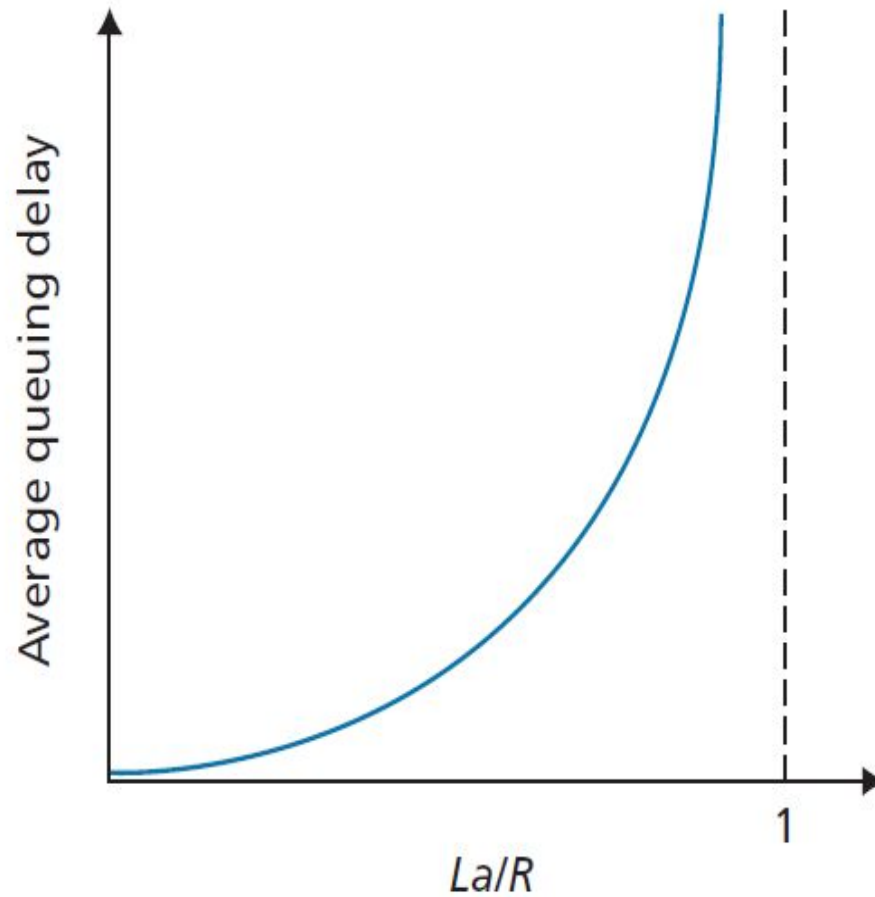


Figure 1.18 ♦ Dependence of average queuing delay on traffic intensity

Packet Loss

In our discussions above, we have assumed that the queue is capable of holding an infinite number of packets.

In reality a queue preceding a link has finite capacity, although the queuing capacity greatly depends on the router design and cost.

Instead, a packet can arrive to find a full queue.

With no place to store such a packet, a router will **drop** that packet; that is, the packet will be **lost**.

The fraction of lost packets increases as the traffic intensity increases.

Therefore, performance at a node is often measured not only in terms of delay, but also in terms of the probability of packet loss.

As we'll discuss in the subsequent chapters, a lost packet may be retransmitted on an end-to-end basis in order to ensure that all data are eventually transferred from source to destination.

Our discussion up to this point has focused on the nodal delay, that is, the delay at a single router.

Let's now consider the total delay from source to destination.

To get a handle on this concept, suppose there are $N - 1$ routers between the source host and the destination host.

Let's also suppose for the moment that the network is uncongested (so that queuing delays are negligible), the processing delay at each router and at the source host is d_{proc} , the transmission rate out of each router and out of the source host is R bits/sec, and the propagation on each link is d_{prop} .

The nodal delays accumulate and give an end-to-end delay,

$$d_{\text{end-end}} = N (d_{\text{proc}} + d_{\text{trans}} + d_{\text{prop}}) \quad (1.2)$$

where, once again, $d_{\text{trans}} = L/R$, where L is the packet size.

Note that Equation 1.2 is a generalization of Equation 1.1, which did not take into account processing and propagation delays.

Traceroute

To get a hands-on feel for end-to-end delay in a computer network, we can make use of the Traceroute program.

Traceroute is a simple program that can run in any Internet host.

When the user specifies a destination hostname, the program in the source host sends multiple, special packets toward that destination.

End System, Application, and Other Delays

In addition to processing, transmission, and propagation delays, there can be additional significant delays in the end systems.